# SIT 218/738: Secure coding

## Credit task 3.2C: Injection attacks-Part 2

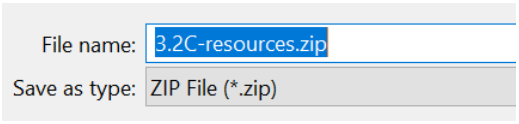
**Objective**

Injection attacks happens due to improper handling of input/untrusted data and their usage in the response which is sent back to the users. In this task you will identify the vulnerability in the given web application and test few injection attacks to exploit its vulnerability.

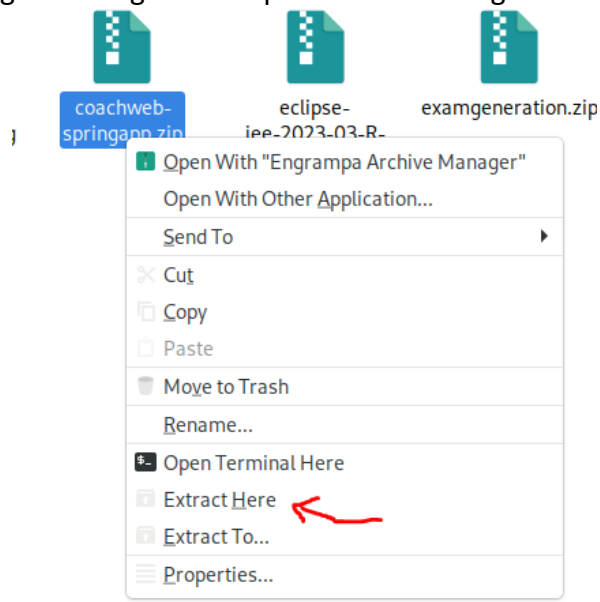
**Overview**

Download the coachweb-springapp from the resources section of Ontrack. You will see a 3.2C-Resources.zip downloaded from Ontrack and save in a separate folder.

**VM Lab: Tasks resources 3.2C-Resources.zip file is also available in the folder /home/kali/Desktop/SIT218/ on the desktop of VM**

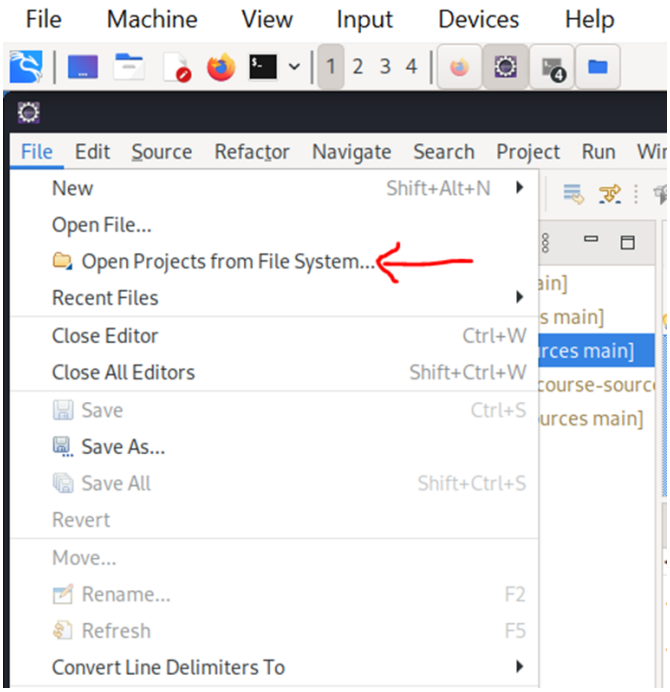



Extract the zip file by right clicking on the zip file and selecting **Extract Here**.

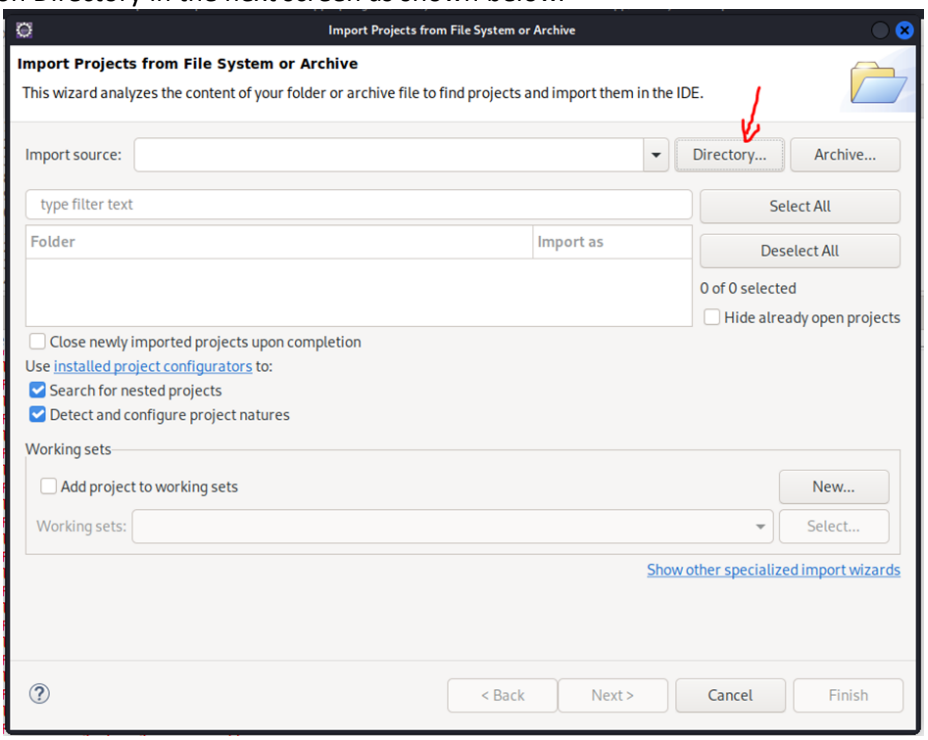



Follow the below steps to import the web app to your Kali-Linux Eclipse IDE. Make sure you are running Eclipse before you do the below steps.

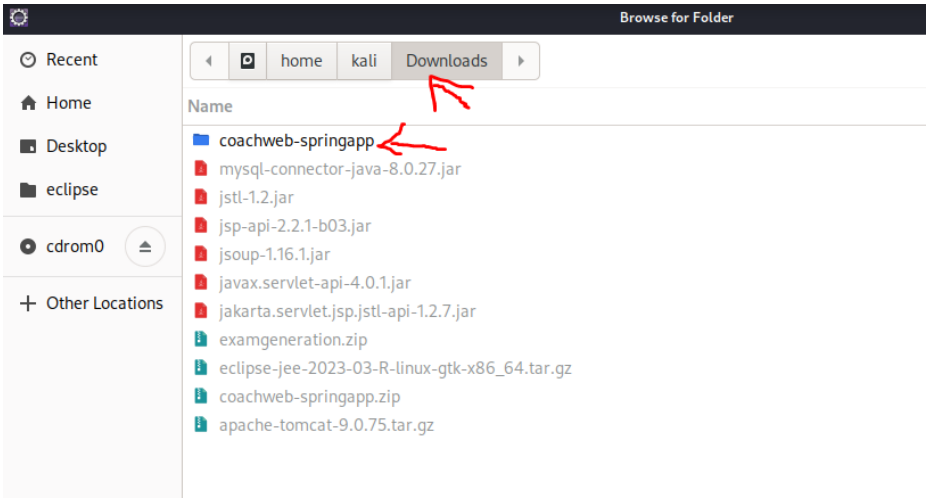
1. In the Eclipse IDE, select **File** > **Open Projects from File System**



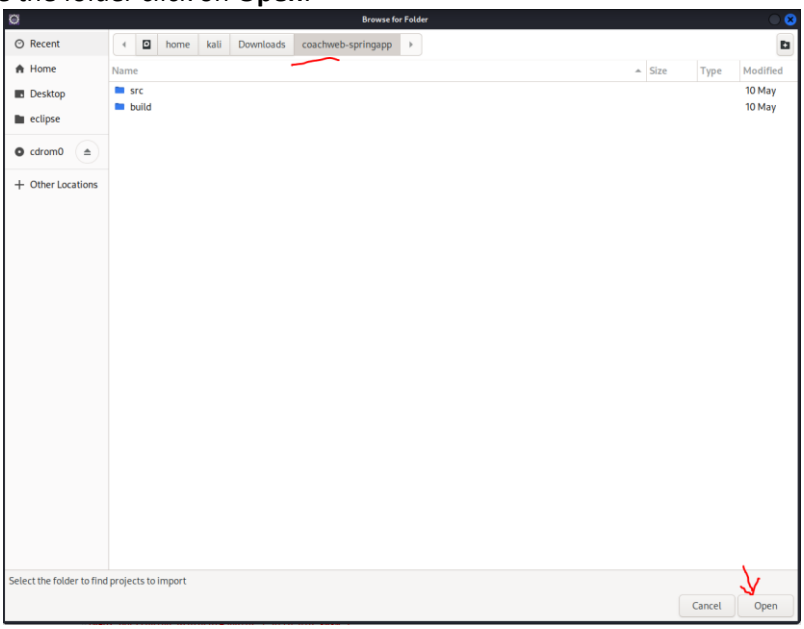

2. Click on Directory in the next screen as shown below.



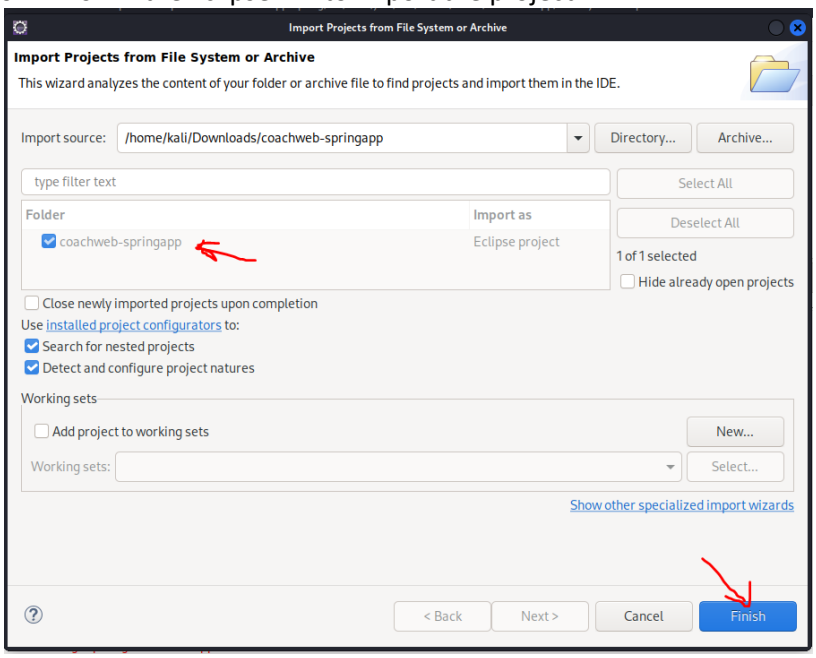

3. Double-click the extracted coachweb-springapp folder in the file system (i.e the files are extracted in the Downloads folder)



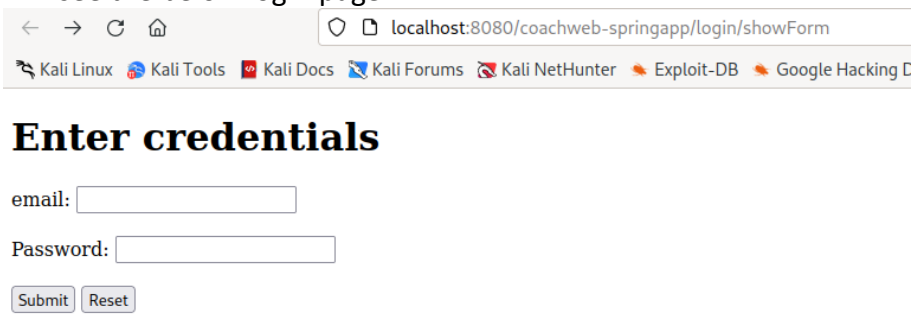

4. Once inside the folder click on **Open**.




5. Now click on **Finish** in the Eclipse IDE to import the project.



6. Now you can run the application on the Tomcat server as done in the class activity.
7. You will see the below login page.




When you login to the web application with the credentials email: bob@gmail.gom and Password: "**Password-1**" you will see a greeting message. When you have an incorrect email id and an incorrect password then you will get an error message.

Task -1: Explain the vulnerability in the code "LoginController.java" class and how can an injection attack be launched against this login page.

```
12 @Controller
13 //
14 public class LoginController {
15
16     @RequestMapping("/login/showForm")
17     public String loginForm(Model model) {
18         model.addAttribute("Login", new Login());
19         return "Login";
20     }
21
22     @PostMapping("/validate")
23     public String processLogin(@ModelAttribute("Login") Login login, Model model) {
24         if(login.getEmailID().equals("bob@gmail.com") & login.getPassword().equals("Password-1"))
25         {
26             model.addAttribute("greeting", "Welcome back");
27             return "result";
28         }
29         else if (!login.getEmailID().equals("bob@gmail.com"))
30             {
31
32             model.addAttribute("err", login.getEmailID() + "is not a registered ID please register first");
33             return "error";
34             }
35         else
36         {
37             model.addAttribute("err","wrong credentials");
38             return "error";
39         }
40     }
41
42 }
```

Task -2: Give three examples of malicious payload that can be injected into this app. Inject your payloads into the appropriate field and execute, show that vulnerability has been exploited for each payload, take screenshots for your submission

**Submission Requirements:**

Submit one PDF file containing the following information:

1. Brief explanation of the vulnerability in the code (highlight the line of code that can be exploited)
2. Three examples of malicious code that can be injected into to the correct field and web app's response (include screenshots)
3. Write a safer error message that will avoid the vulnerability when email address is not in the list.

**Submission Due**

The due for each task has been stated via its OnTrack task information dashboard.